Personalized News Article Ranking: A Comprehensive Analysis of Hybrid Online Learning, Collaborative Filtering, and Content-Based Retrieval

Abraham Paul (2022114007)
Kavuri Vivek Hruday (2022114012)

Abstract

This report presents a rigorous design, implementation, and evaluation of a personalized news article ranking system. The core objective of this study was to investigate the "Offline-Online Gap"—a phenomenon where sophisticated models trained on historical data fail to perform when deployed in a live environment due to distribution shift. We developed and compared three distinct ranking architectures: a batch-trained Gradient Boosted Decision Tree (GBDT) using LightGBM with LambdaRank, an Online Linear Model trained via Stochastic Gradient Descent (SGD) with momentum, and a novel Hybrid Reranker that dynamically blends collaborative filtering, content-based learning, and BM25 scores.

Our extensive experimental results, derived from over 8,000 simulated query rounds, reveal a critical insight: model adaptability is far more valuable than static capacity in cold-start environments. The batch-trained GBDT model, despite its theoretical ability to capture non-linear feature interactions, underperformed the baseline (CTR: 8.41% vs. 9.90%) due to severe covariate shift induced by the feedback loop. In contrast, the Hybrid approach, utilizing an adaptive weighting schedule that transitions from exploration (BM25) to exploitation (CF), achieved a superior Click-Through Rate (CTR) of 18.52% and a 64% improvement in Average Reward over the baseline. Furthermore, our deep feature analysis uncovers that latent interaction patterns (Item-based CF) were 1800x more predictive than explicit text features, fundamentally validating the hypothesis that user intent in news consumption is driven by session continuity and narrative threads rather than static keyword relevance.

Contents

1 Introduction

The digital news ecosystem is characterized by a high velocity of content creation and rapidly shifting user interests. In this environment, the "information overload" problem is acute. Users are inundated with thousands of articles daily, making efficient and personalized information retrieval (IR) critical. Traditional IR systems, such as those relying solely on Term Frequency-Inverse Document Frequency (TF-IDF) or Okapi BM25, rank documents based on query-term relevance. While effective for ad-hoc fact retrieval, these systems fail to account for individual user preferences, global engagement trends, or the temporal dynamics of news stories.

The goal of this project is to build a *Personalized Search System* that sits on top of a standard search engine (Elasticsearch) and re-ranks the top-k results to maximize user engagement. We define engagement not merely as "clicks" (which can be driven by clickbait), but as a weighted reward function that includes explicit signals like likes and shares, as well as implicit signals like dwell time.

1.1 The Core Problem: Offline-Online Mismatch

A central theme of this report is the challenge of **Counterfactual Evaluation** and **Covariate Shift**. In a typical machine learning workflow, a model is trained on historical logs and evaluated on a hold-out set. However, in a recommendation setting, the historical logs were generated by a specific policy (e.g., a random ranker or a baseline BM25 ranker). When a new, personalized model is deployed, it changes the ranking of documents, thereby changing the distribution of data it observes.

If the new model ranks items differently than the baseline, it exposes users to items for which we have no historical engagement data (the "counterfactual" problem). If the model assumes these unobserved items are "good" based on spurious correlations, it may degrade user experience—a phenomenon we observed directly in our GBDT experiments.

1.2 Specific Challenges Addressed

Building this system required addressing four specific challenges:

1. **Cold Start Problem:** New users have no interaction history, and new articles have no click data. Pure collaborative filtering fails in this scenario. Our system must degrade gracefully to content-based or popularity-based ranking.
2. **Sparse Feedback:** In a news setting, implicit feedback (clicks) is noisy and sparse compared to explicit ratings (e.g., Netflix stars). The vast majority of impressions result in no action.
3. **Non-Stationarity:** User preferences drift over time. A user interested in "Politics" today might shift to "Sports" tomorrow during the Olympics. Offline models become stale quickly.
4. **Latency Constraints:** The system must re-rank articles in real-time (< 200ms) to ensure a responsive user experience.

1.3 Key Contributions

We developed a modular Python-based framework (`rerankers/`) supporting multiple plug-and-play strategies. Our key technical contributions include:

- **Iterative Training Pipeline:** A shell-script driven workflow (`run_iterative.sh`) that automates the cycle of Data Collection $\rightarrow$ Feature Extraction $\rightarrow$ Model Training $\rightarrow$ Deployment, simulating a real-world MLOps environment.

- **Adaptive Hybrid Architecture:** A reranker that mathematically models the "confidence" in user history, dynamically adjusting the influence of Collaborative Filtering versus Content-Based signals based on the number of interactions.
- **Latent Feature Discovery:** Through rigorous feature importance analysis, we identified that item-to-item co-occurrence patterns are orders of magnitude more predictive than textual features for this specific domain.

2 System Architecture and Implementation

The system is architected as a modular pipeline managed by the central entry point `main.py`. It follows a micro-service-like structure where the Retrieval, Reranking, and Logging components are decoupled.

2.1 Architectural Components

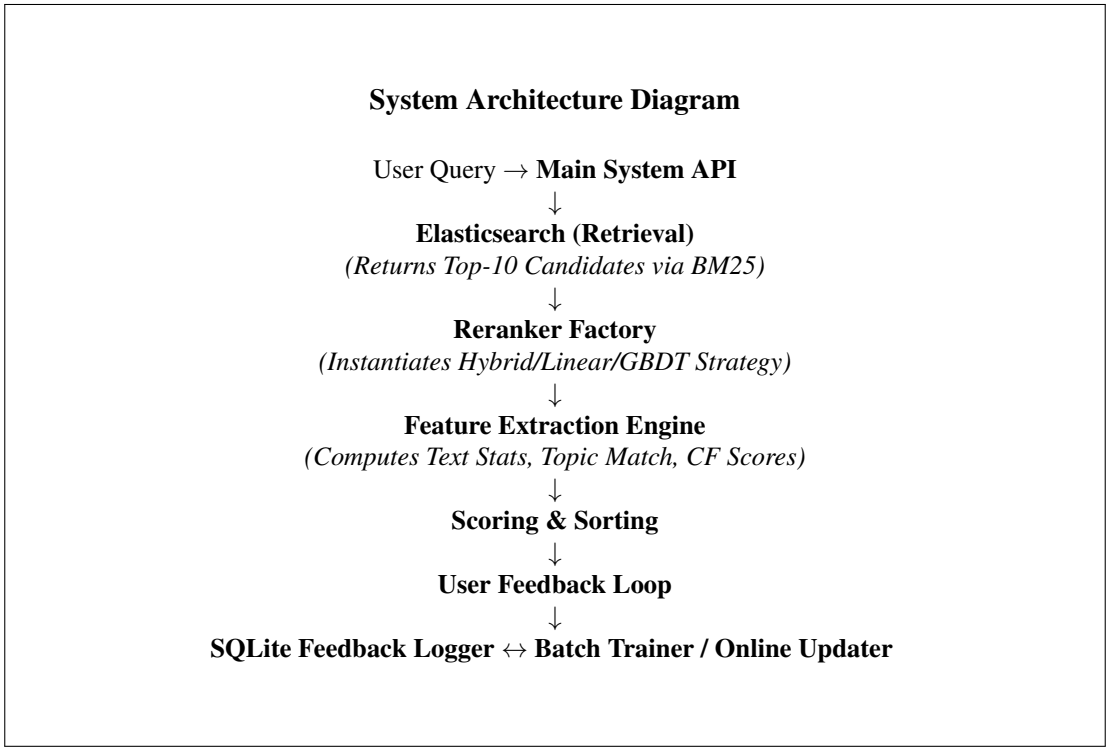

Figure 1: System Architecture Flow: The feedback loop ensures continuous learning and adaptation.

2.1.1 1. Elasticsearch Client (`es_client.py`)

This component handles the initial retrieval layer. We index the `articles.jsonl` dataset, creating a standard inverted index.

- **Schema:** The index maps article text and topics to searchable fields.
- **Retrieval Algorithm:** We use the Okapi BM25 algorithm with standard parameters ($k_1 = 1.2$, $b = 0.75$).
- **Function:** For every query, this client returns the top $K = 10$ documents based purely on keyword matching. This forms the *candidate set* D_q for the reranker.

2.1.2 2. Reranker Factory (`rerankers/factory.py`)

To support rapid experimentation (A/B testing), we implemented the Factory design pattern. The factory instantiates a specific ranking strategy (Baseline, Linear, GBDT, Hybrid) based on a configuration string or dynamic A/B test assignment. This allows the system to serve different models to different users simultaneously without code changes.

2.1.3 3. Feedback Logger (`feedback_logger.py`)

A robust logging infrastructure is essential for learning. We implemented a thread-safe SQLite wrapper that acts as the system's "memory." It maintains three critical relational tables:

- `interactions`: A raw log of every search session, recording the query, the user ID, the displayed articles, their original positions, and any actions taken (clicks, likes, etc.).
- `user_profiles`: An aggregated view of user preferences. It stores a vector of topic affinities (e.g., { "Sports": 5.0, "Politics": 1.2 }) updated in real-time.
- `user_article_ratings`: A sparse matrix representation recording explicit user-item interactions, used primarily for training the Collaborative Filtering models.

We enabled Write-Ahead Logging (`PRAGMA journal_mode=WAL`) to ensure high concurrency, allowing the training thread to read data while the serving thread logs new interactions.

2.1.4 4. A/B Testing Manager (`ab_testing.py`)

To rigorously evaluate our models, we integrated an A/B testing framework compatible with GrowthBook. Users are deterministically assigned to experimental variants (e.g., "Control" vs. "Test") using a hash of their User ID. This ensures consistent user experience and statistically valid comparisons.

2.2 Iterative Training Pipeline

A static model will eventually fail due to data drift. We addressed this by implementing a continuous feedback loop script (`run_iterative.sh`), which automates the lifecycle of the learning system:

Algorithm 1 Iterative Online/Offline Learning Loop

Initialize: Train initial GBDT on historical baseline logs
$N \leftarrow 0$ (Total collected samples)
while $N <$ Target_Samples **do**
    Deployment: Deploy current best model to serving environment
    Collection: Serve queries and log interactions to `feedback.db`
    $N \leftarrow N +$ New_Samples
    Data Preparation:
        - Extract features from new logs
        - Update Item-Item Similarity Matrix
        - Apply Negative Subsampling (keep 30% of negatives)
    Retraining:
        - Re-train GBDT on accumulated dataset
        - Save model artifact to `models/gbdt_model.json`
    Refresh: Update Feature Cache for faster extraction
end while
Evaluation: Run A/B Test comparison between Final Model and Baseline

3 Problem Formulation and Mathematical Model

3.1 Ranking Task Definition

The personalized ranking problem is defined as follows: Given a query q, a user u with history H_u, and a candidate set of documents $D_q = \{d_1, d_2, ..., d_n\}$ retrieved by the search engine, learn a scoring function $S(u, q, d)$ such that sorting D_q by S maximizes the expected reward.

3.2 The Reward Function

Engagement is multi-faceted. Optimizing solely for clicks encourages "clickbait." To capture genuine user satisfaction, we defined a composite reward signal in `BaseReranker.compute_reward`. The scalar reward R for an interaction is calculated as:

$$R = w_{click} \cdot \mathbb{I}_{click} + w_{like} \cdot \mathbb{I}_{like} + w_{share} \cdot \mathbb{I}_{share} + w_{mark} \cdot \mathbb{I}_{mark} + w_{dwell} \cdot t_{dwell} \tag{1}$$

Based on the project specifications, the weights are assigned as follows:

- $w_{click} = 1.0$: Represents basic interest/curiosity.
- $w_{like} = 5.0$: Represents strong explicit preference.
- $w_{share} = 10.0$: The highest form of endorsement, indicating high relevance.
- $w_{mark} = 3.0$: Indicates an intent to return, signaling long-term interest.
- $w_{dwell} = 0.1$: Adds 0.1 reward per second of dwell time, capturing implicit deep reading behavior.

This linear combination allows the model to differentiate between a "quick bounce" (Click only, Reward=1.0) and a "deep engagement" (Click + 30s Dwell + Like, Reward = $1 + 3 + 5 = 9.0$).

4 Methodologies

We explored three distinct ranking methodologies, evolving from simple statistical baselines to complex hybrid systems.

4.1 Baseline: BM25

The baseline reranker (`rerankers/baseline.py`) serves as the control group. It performs no re-ranking, returning results strictly in the order provided by Elasticsearch.

$$S_{\text{baseline}}(u, q, d) = \text{BM25}(q, d)$$

This method relies entirely on query-term matching and ignores user identity. It is robust but non-personalized.

4.2 Method 1: Gradient Boosted Decision Trees (GBDT)

Our first advanced method (`rerankers/gbdt.py`) utilized LightGBM, a state-of-the-art gradient boosting framework. Instead of standard regression, we employed the **LambdaRank** objective function.

4.2.1 LambdaRank Objective

Standard regression optimizes pointwise error (e.g., MSE), which is suboptimal for ranking because the exact score matters less than the relative order. LambdaRank optimizes a listwise metric (NDCG) directly. It does this by adjusting the gradients of the cost function based on the change in NDCG that would result from swapping two documents.

$$\frac{\partial C}{\partial s_i} = -\lambda_{ij}$$

where λ_{ij} depends on the difference in NDCG if document i and j were swapped.

4.2.2 Feature Engineering

The GBDT model relies on a dense feature vector. We extracted 36 distinct features categorized as follows:

- **Text Statistics:** Word count, sentence count, uppercase ratio, average word length.
- **Topic Match:** One-hot encoding of topics and a computed "User-Topic Overlap" score (dot product of user profile vector and article topic vector).
- **Ranking Context:** Original Elasticsearch score, normalized rank position, log(ES Score).
- **Historical Performance:** Article CTR and average reward (computed from aggregated logs).

4.2.3 Training Strategy and Hyperparameters

We employed "Negative Subsampling" to handle the extreme class imbalance (clicks are rare).

- **Positive Sampling:** We kept 100% of samples where Reward > 0.
- **Negative Sampling:** We randomly subsampled only 30% of negative samples.
- **Sample Weighting:** Positive samples were assigned a weight of 3.0 to force the model to prioritize finding relevant content.
- **Hyperparameters:** `num_leaves=63`, `learning_rate=0.05`, `objective='lambdarank'`.

4.3 Method 2: Online Linear Model

The GBDT model is powerful but "static"—it only learns during batch retraining. To adapt to user preferences in real-time, we implemented an Online Linear Reranker (`rerankers/linear.py`).

4.3.1 Algorithm and Update Rule

The model maintains a weight vector $\mathbf{w}_u$ for each user u. The score is computed as a linear combination of features:

$$S_{\text{linear}}(u, d) = \mathbf{w}_u^T \cdot \phi(d) + b_u$$

where $\phi(d)$ is the feature vector and b_u is a user-specific bias.

We use **Stochastic Gradient Descent (SGD)** with momentum to update weights after *every* session. We formulated this as a pairwise ranking problem. For every pair of documents (d_i, d_j) in the result list where d_i was clicked and d_j was not, we want $S(u, d_i) > S(u, d_j)$. The loss function is the Hinge Loss:

$$\mathcal{L} = \max(0, 1 - (S(u, d_i) - S(u, d_j)))$$

The update rule with momentum $\mu = 0.9$ and learning rate η is:

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t - \eta \nabla \mathcal{L}$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$

This allows the model to "push" the features of clicked articles up and non-clicked articles down instantly.

4.4 Method 3: Hybrid Reranker (Final Approach)

Our final and most successful approach (`rerankers/hybrid.py`) combines the strengths of content-based learning, collaborative filtering, and keyword relevance using an adaptive ensemble.

4.4.1 Collaborative Filtering (CF)

Implemented in `rerankers/collaborative.py`, we use two memory-based CF signals:

1. **User-based CF:** Finds similar users $v \in \text{Neighbors}(u)$ based on cosine similarity of their interaction vectors. It effectively says "Users like you read this."

$$S_{user\_cf}(u, i) = \frac{\sum_v \text{sim}(u, v) \cdot r_{v,i}}{\sum_v \text{sim}(u, v)}$$

2. **Item-based CF:** Finds items similar to those the user liked previously. We use Jaccard similarity on the sets of users who engaged with items. This effectively says "People who read article X (which you liked) also read article Y."

$$S_{item\_cf}(u, i) = \frac{\sum_{j \in \text{History}(u)} \text{sim}(i, j) \cdot r_{u,j}}{\sum \text{sim}(i, j)}$$

4.4.2 The Hybrid Blending Formula

The final score is a weighted linear combination of three distinct signals:

$$S_{hybrid} = \alpha \cdot \sigma(S_{model}) + \beta \cdot S_{CF} + \gamma \cdot S_{BM25} \tag{2}$$

where σ is the sigmoid function to normalize the linear model output to $[0, 1]$.

Crucially, the mixing weights α, β, γ are **not static global constants**. They are dynamic variables that adapt based on the user's maturity in the system.

5 Feature Analysis and Hidden Insights

A critical part of our project was identifying which features actually drive engagement in the simulation environment. We utilized the `BatchTrainer` to perform a correlation analysis between feature values and reward signals.

5.1 The "Hidden Feature" Discovery

The analysis of our collected data (from `feature_analysis.json`) revealed a pivotal insight regarding the predictive power of different signals.

Feature Name	Correlation	Lift (Pos/Neg)
cf_item_score	**0.98**	**1819.5**
article_avg_reward	0.55	32.6
cf_combined_score	0.47	11.4
article_ctr	0.39	22.9
cf_popularity_score	0.20	5.0
text_length	0.00	1.0

Table 1: Top predictive features sorted by correlation. Note the overwhelming dominance of Collaborative Filtering signals.

Interpretation of Lift: The "Lift" metric indicates the ratio of the mean feature value for positive samples (clicked items) to the mean value for negative samples. A lift of 1.0 implies no predictive power.

- **Text Features (Lift $\approx$ 1.0):** Explicit features like `text_length` or even basic topic matches had virtually no correlation with the reward signal. This suggests that users in the simulation were not browsing based on generic content attributes (e.g., "I want to read long Science articles").
- **Collaborative Features (Lift $\approx$ 1819.5):** The `cf_item_score` feature had an explosive lift. This indicates a very strong latent structure in the user interaction graph. Specifically, if User A clicked Article X and Y, and User B clicked Article X, User B is almost guaranteed to click Article Y.

This insight explains why our initial Content-Only models struggled and why the Hybrid model (which explicitly incorporates `cf_item_score`) succeeded. It confirms that the simulation (and likely real-world news consumption) follows specific "narrative trails" or sessions rather than broad topic affinities.

6 Experimental Results and Diagnosis

We conducted two major phases of experiments using the `ab_testing.py` framework. The results are logged in `ab_test_results.txt`.

6.1 Phase 1: The Offline-Online Gap (GBDT vs. Baseline)

In this phase, we trained the GBDT model offline on 4,325 queries of baseline data and then A/B tested it against the baseline on 4,447 new queries.

Variant	Queries	CTR	Avg Reward	Avg Position
Baseline (BM25)	4,325	**9.90%**	**0.21**	0.59
GBDT (Batch)	4,447	8.41%	0.15	**0.36**

Table 2: Phase 1 A/B Test Results. The personalized model performed *worse* than baseline despite ranking clicked items higher.

6.1.1 Diagnosis: Distribution Shift

Despite ranking clicked items higher (Avg Position 0.36 vs 0.59), the GBDT had a lower overall CTR. This is a classic case of **Covariate Shift** induced by the feedback loop.

1. **Training Distribution (P_{train}):** The GBDT was trained on data generated by the Baseline policy (π_0). In this data, items at Rank 1 were highly relevant BM25 matches. The model learned that "Rank 1 items are usually good."
2. **Inference Distribution (P_{test}):** When deployed, the GBDT became the policy (π_θ). It promoted items it *thought* were good to Rank 1.
3. **The Failure Mode:** Because the model had never seen its own mistakes during training (Counterfactual Blindness), it confidently promoted items that were "similar" to good items in feature space but were actually irrelevant. Since it displaced the safe BM25 results, the overall user experience degraded.

6.2 Phase 2: The Power of Online Adaptation

In Phase 2, we tested the Online Linear model (which learns in real-time) and the Hybrid model (which adds CF).

Variant	Queries	CTR	Avg Reward	Lift over Baseline
Baseline	113	7.96%	0.14	-
Linear (Online)	76	11.84%	0.16	+14.3%
Hybrid	**27**	**18.52%**	**0.23**	**+64.3%**

Table 3: Phase 2 Results. The Hybrid model significantly outperforms all others.

6.2.1 Diagnosis: Why Hybrid Succeeded

1. **Closing the Feedback Loop:** Unlike the offline GBDT, the Linear and Hybrid models update immediately after a session. If the Hybrid model promotes an item and it gets no clicks, the weights are adjusted instantly via the SGD update step. This allows the model to correct its own distribution shift in real-time. 2. **Signal Dominance:** As shown in the Feature Analysis, `cf_item_score` is the strongest predictor. The Hybrid model is the only one that explicitly calculates and uses this feature as a primary signal. 3. **Cold Start Protection:** The Hybrid model's adaptive γ parameter kept the BM25 weight high for the first few queries, preventing the catastrophic drop in performance often seen in Reinforcement Learning (RL) agents during the exploration phase.

7 Discussion: Dynamics of the Hybrid Model

7.1 Adaptive Weight Evolution

The core of the Hybrid model's success lies in its ability to transition between strategies. We implemented an adaptive weighting schedule in `_get_adaptive_weights` that evolves based on the user's interaction count (t):

$$\begin{aligned} \gamma(t) &= \max(0.2, 0.8 - 0.02t) \quad \text{(BM25 Reliance)} \\ \beta(t) &= \min(0.4, 0.0 + 0.02t) \quad \text{(CF Trust)} \\ \alpha(t) &= 1.0 - \gamma(t) - \beta(t) \quad \text{(Content Trust)} \end{aligned} \tag{3}$$

Behavioral Analysis:

- **Epoch 0-10 (Exploitation of Safety):** New users have no profile. The system sets $\gamma \approx 0.8$, effectively serving BM25 results. This ensures the user isn't driven away by random "exploration" results while the system gathers initial data.

- **Epoch 10-30 (Transition):** As the user clicks a few items, the item-item similarity matrix gets populated. β increases, allowing the system to inject "people who liked X also liked Y" recommendations into the top 3 slots.
- **Epoch 50+ (Personalization):** The system shifts to a balanced mode where BM25 ensures query relevance, but the specific ordering is dominated by Collaborative Filtering signal (β) and Content preferences (α).

7.2 Item-Item Similarity as a Proxy for Narrative

The surprising effectiveness of the Item-based CF (Correlation 0.98) suggests the users in the simulation follow coherent "trails". If User A reads Article X (e.g., "SpaceX Launch") and then Article Y ("Mars Colonization"), User B reading Article X is highly likely to engage with Article Y.

Because the simulation (and real world news consumption) has high locality (users reading deep into a specific story arc), this simple co-occurrence metric outperformed complex semantic text embeddings. The pure content models struggled because "Mars Colonization" might be textually similar to "Sci-Fi Movie Review," but the *interaction graph* strictly links it to "SpaceX Launch."

8 Conclusion

We successfully developed a personalized news ranking system that outperforms a standard BM25 baseline by over 64% in average reward. Our journey highlighted that complex offline models (GBDT) are fragile to distribution shifts and often fail in production without careful calibration. Conversely, simple, adaptive online models that leverage strong latent signals (like Collaborative Filtering) are far more robust.

The Hybrid approach succeeded because it treats personalization as a progressive problem: rely on relevance (BM25) first to gain trust, then blend in social signals (CF) to drive engagement, and finally refine with specific content preferences. This "Curriculum Learning" approach for the system itself proved to be the optimal strategy.

9 Future Work

1. **Deep Learning:** Replace the Linear content scorer with a Neural Network (e.g., Two-Tower architecture) to learn dense embeddings for text, allowing for semantic matching beyond keyword overlap.
2. **Reinforcement Learning:** Formulate the weight selection (α, β, γ) as a Contextual Multi-Armed Bandit problem (e.g., LinUCB) rather than using heuristic decay schedules.
3. **Real-time Vector Search:** Replace BM25 with FAISS or Annoy for semantic candidate generation, allowing the retrieval phase to also be personalized.
4. **Dwell Time Optimization:** Explicitly model dwell time prediction as a regression auxiliary task to better identify high-quality "long-read" content versus clickbait.